INFO 5 – Projet 2 : Cryptographie élémentaire

Niveau : L1 Maths | Séance 3 sur 4 (3h) : L'Analyse Fréquentielle

Objectifs de la séance

- Comprendre les propriétés statistiques d'une langue (fréquence des lettres).
- Manipuler les dictionnaires Python pour compter des occurrences.
- Générer un affichage graphique en console (Histogramme).
- Implémenter un algorithme de recherche de maximums multiples.
- Casser un chiffrement mathématiquement, **sans tester toutes les clés**.

Partie 1 : Les statistiques et l'Histogramme

En français, la lettre **E** est la plus fréquente (environ 15% à 17% d'un texte normal). Dans un texte chiffré par décalage, la lettre la plus fréquente sera donc très probablement le 'E' décalé.

Exercice 1 : Compter les lettres

Écrivez une fonction qui parcourt un texte et compte combien de fois chaque lettre apparaît, en utilisant un dictionnaire Python. *Ignorez la ponctuation et les espaces.*

```python
def compter_lettres(texte):
    """Retourne un dictionnaire avec le nombre d'apparitions de chaque lettre."""
    comptes = {}
    texte_minuscule = texte.lower()


    return comptes
```

Exercice 2 : Visualisation (L'Histogramme)

Pour comprendre la répartition des lettres d'un texte intercepté, les cryptanalystes tracent des graphiques. Écrivez une fonction qui affiche un histogramme horizontal dans la console à l'aide du caractère `*` répété.

```python
def afficher_histogramme(comptes):
    """Affiche un graphique en barres des occurrences."""
    print("\n--- Histogramme des frequences ---")



        print(f"{lettre} | {barre} ({score})")
```

Partie 2 : L'attaque naïve par le Maximum

Exercice 3 : Trouver la lettre reine

Écrivez une fonction `lettre_plus_frequente(comptes)` qui parcourt le dictionnaire (avec `.items()`) et renvoie la lettre qui a le plus grand score.

Exercice 4 : L'attaque statistique (Calcul de la clé)

Pour trouver le décalage, il suffit de faire la différence entre la position de la lettre chiffrée trouvée et la position de la lettre 'e' (en utilisant `ord()`).

```
def attaque_frequentielle_simple(texte_chiffre):

    return dechiffrer_cesar(texte_chiffre, cle)
```

Partie 3 : L'attaque Robuste (Le Top 3)

Que se passe-t-il si, dans notre texte, le **A** ou le **S** est exceptionnellement plus présent que le E ? L'attaque précédente va échouer. Pour créer un outil de qualité professionnelle, l'algorithme doit extraire les **3 lettres les plus fréquentes**, générer les 3 clés correspondantes (en supposant pour chacune qu'elle correspond au 'e'), et proposer les 3 déchiffrements à l'utilisateur.

Exercice 5 : Extraire le Top 3

Écrivez une fonction qui trouve les 3 lettres les plus fréquentes sans utiliser de fonctions de tri toutes faites. *Indice : Cherchez le maximum, ajoutez-le à votre liste des gagnants, puis forcez son score à 0 dans le dictionnaire pour pouvoir chercher le "nouveau" maximum à l'étape suivante.*

```
def trouver_top_3(comptes):
    """Retourne une liste des 3 lettres les plus frequentes."""
    # On fait une copie du dictionnaire pour pouvoir le modifier (mettre a 0)
    comptes_copie = comptes.copy()
    top_lettres = []

    # On repete l'operation 3 fois
    for i in range(3):


    return top_lettres
```

Exercice 6 : L'attaque multi-cibles

Utilisez votre `trouver_top_3` pour coder `attaque_frequentielle_robuste(texte_chiffre)`. La fonction doit calculer 3 clés différentes, déchiffrer le texte 3 fois, et afficher les 3 propositions. Testez-la sur le message fourni par votre enseignant !

Partie 4 : Arithmétique Modulaire

Le chiffrement affine utilise une multiplication et une addition. La formule est :

$$E(x) = (a \cdot x + b) \pmod{26}$$

Condition vitale : Pour que le déchiffrement soit possible, il faut absolument que a et 26 soient **premiers entre eux** (leur PGCD doit être égal à 1).

Exercice 7 : L'Algorithme d'Euclide

Écrivez une fonction `pgcd(a, b)` qui calcule le Plus Grand Commun Diviseur à l'aide d'une boucle `while`.

```
def pgcd(a, b):
    """Calcule le PGCD avec l'algorithme d'Euclide."""
    while b != 0:
        # A completer
        # ...
    return a
```

Exercice 8 : L'Inverse Modulaire

Pour déchiffrer, il nous faut l'inverse modulaire de a modulo 26, noté a^{-1}, tel que :

$$(a \cdot a^{-1}) \pmod{26} = 1$$

Testez tous les nombres de 1 à 25 pour le trouver.

```
def inverse_modulaire(a, m=26):
    """Trouve l'inverse de 'a' modulo 'm' par recherche exhaustive."""
    if pgcd(a, m) != 1:
        return None # Pas d'inverse possible

    for i in range(1, m):
        if (____ * ____) % m == 1:
            return i
    return None
```

Partie 5 : Chiffrement et Déchiffrement Affine

Exercice 9 : Chiffrer

Écrivez la fonction `chiffrer_affine(texte, a, b)`. *Rappel : Utilisez `ord()` et `chr()` pour passer des lettres aux nombres (A=0, B=1...).*

Exercice 10 : Déchiffrer

La formule mathématique du déchiffrement est :

$$D(y) = a^{-1} \cdot (y - b) \pmod{26}$$

Implémentez `dechiffrer_affine(texte_chiffre, a, b)` en utilisant votre fonction `inverse_modulaire`.

Partie 6 : L'Attaque en Force Brute sur l'Affine (bonus)

Partie 6 (bonus) : Cette partie est facultative. Elle s'adresse aux étudiants ayant terminé les parties précédentes. Elle ne sera pas pénalisante si elle n'est pas réalisée.

Le chiffrement affine possède 26 possibilités pour b. Pour a, il n'y a que 12 nombres premiers avec 26 (1, 3, 5, 7, 9, 11, 15, 17, 19, 21, 23, 25). Le nombre total de clés est de $12 \times 26 = 312$.

Exercice 11 : Casser le code

Écrivez une fonction `attaque_affine_automatique(texte_chiffre)` qui teste les 312 clés avec deux boucles `for` imbriquées. Elle devra filtrer les résultats et s'arrêter si elle trouve au moins 2 mots de la liste `[" le ", " la ", " de ", " et ", " est "]`.